



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени
Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «ИУ Информатика и системы управления»

КАФЕДРА «ИУ7 Программное обеспечение ЭВМ и информационные технологии»

Лабораторная работа № 2 по дисциплине «Анализ алгоритмов»

Тема Сравнительный анализ рекурсивного и нерекурсивного алгоритмов

Студент Еланская Е.Д.

Группа ИУ7-53Б

Преподаватели Волкова Л.Л., Строганов Ю.В.

Москва, 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Аналитическая часть	4
1.1 Задание по варианту	4
2 Конструкторская часть	5
2.1 Описание алгоритмов	5
2.2 Расчёт трудоёмкости алгоритмов	6
3 Технологическая часть	7
3.1 Реализация алгоритмов	7
3.2 Тестирование	7
4 Исследовательская часть	8
4.1 Замеры процессорного времени	8
ЗАКЛЮЧЕНИЕ	9
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	10

ВВЕДЕНИЕ

Цель работы – провести сравнительный анализ рекурсивного и нерекурсивного алгоритмов.

Задачи, необходимые для достижения поставленной цели:

- 1) разработать рекурсивный и нерекурсивный алгоритмы решения задачи, согласно варианту;
- 2) описать средства разработки и инструменты замера процессорного времени выполнения реализации алгоритмов;
- 3) реализовать разработанные алгоритмы;
- 4) выполнить тестирование реализации алгоритмов;
- 5) выполнить теоретическую оценку затрачиваемой реализацией каждого алгоритма памяти;
- 6) выполнить замеры процессорного времени выполнения реализации алгоритмов в зависимости от варьируемого входа;
- 7) оценить трудоёмкость двух алгоритмов или их реализаций в худшем случае;
- 8) сравнить результаты замеров процессорного времени и оценки трудоёмкости;
- 9) сделать выводы из сравнительного анализа реализации рекурсивного и нерекурсивного алгоритмов решения одной и той же задачи, заданной вариантом, по критериям ёмкостной эффективности.

1 Аналитическая часть

Рекурсивный алгоритм – алгоритм, который вызывает сам себя, пока не достигнет условия остановки (базового условия) [1].

1.1 Задание по варианту

Выведите все целые числа от 1 до N включительно в порядке возрастания.

Вывод

В данном разделе мной был проведён анализ теоретических материалов, объясняющих принцип работы рекурсивных алгоритмов.

2 Конструкторская часть

2.1 Описание алгоритмов

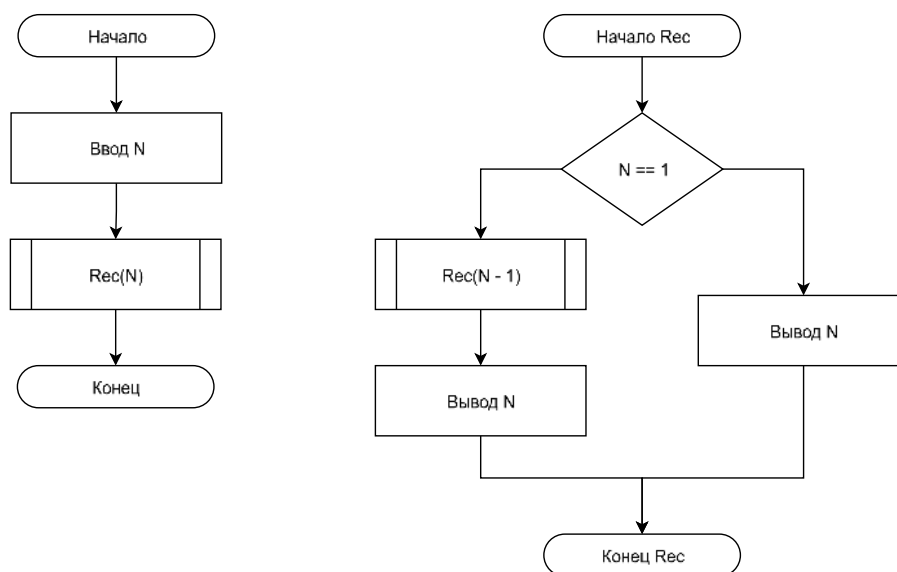


Рисунок 2.1 — Схема рекурсивного алгоритма.

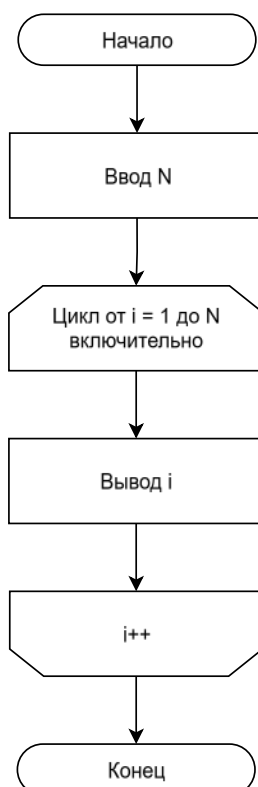


Рисунок 2.2 — Схема нерекурсивного алгоритма.

2.2 Расчёт трудоёмкости алгоритмов

Введём модель вычислений для определения теоретической трудоёмкости описанных ранее алгоритмов.

Таблица 2.1 — Цены базовых операций.

Операции	Цена
$+, -, =, ++, --, \ll, \gg$	1
$*, /, \%, *=, /=$	2
$==, >=, <=, !=, >, <, \&\&, \parallel$	1
$[], \{\}$	1
call/return	34

Цена вызова-возврата рассчитывалась по формуле 2.1:

$$CR = 2 \cdot (p + k + r + l + f), \quad (2.1)$$

где

p - количество передаваемых параметров,

k - количество сохраняемых в стеке регистров,

r - количество возвращаемых по адресной ссылке значений,

l - количество локальных ячеек для обеспечения реентерабельности (повторного вызова до завершения предыдущего),

f - количество ячеек для возвращаемого значения.

Таблица 2.2 — Трудоёмкость алгоритмов.

Алгоритм	Трудоёмкость ХС
Рекурсивный	$(2N - 1) CR + 3N$
Нерекурсивный	$CR + 2 + 3N$

Вывод

В данном разделе мной были построены схемы алгоритмов и вычислена их трудоёмкость с помощью введённой модели вычислений.

3 Технологическая часть

3.1 Реализация алгоритмов

Для реализации алгоритмов использовался язык программирования C++, так как он поддерживает статическую типизацию и имеет возможность отключения сбойщика мусора. Отладка происходила в среде разработки «Visual Studio 2022».

```
1 void print_n_rec(int n){
2     if (n == 1)
3         std::cout << n << " ";
4     else{
5         print_n(n - 1);
6         std::cout << n << " ";
7     }
8 }
```

Листинг 1 – Рекурсивный алгоритм.

```
1 void print_n(int n){
2     for (int i = 1; i <= n; i++)
3         std::cout << i << " ";
4 }
```

Листинг 2 – Нерекурсивный алгоритм.

3.2 Тестирование

Таблица 3.1 — Модульные тесты для рекурсивного алгоритма.

Вход	Выход
1	1
10	1 2 3 4 5 6 7 8 9 10

Таблица 3.2 — Модульные тесты для нерекурсивного алгоритма.

Вход	Выход
1	1
10	1 2 3 4 5 6 7 8 9 10

Вывод

В данном разделе мной были реализованы рекурсивная и нерекурсивная версии алгоритмов по варианту и проведено их успешное тестирование.

4 Исследовательская часть

4.1 Замеры процессорного времени

Замеры проводились на ноутбуке Redmibook Pro 14 2022:

- процессор Intel i7-12650H (архитектура x86-64, 10 ядер по 2.3 ГГц);
- оперативная память 16 Гб (DDR4, 5200 МГц);
- операционная система Windows 11 Home 25H2.

Для замеров использовалась функция `clock()` из библиотеки `ctime` [2, с. 288].

Таблица 4.1 — Некоторые замеры процессорного времени для различных N (при $k = 1000$).

N	Рекурсивный	Нерекурсивный
1	0.067	0.051
10	0.678	0.717
20	1.523	1.912
30	2.081	2.1885

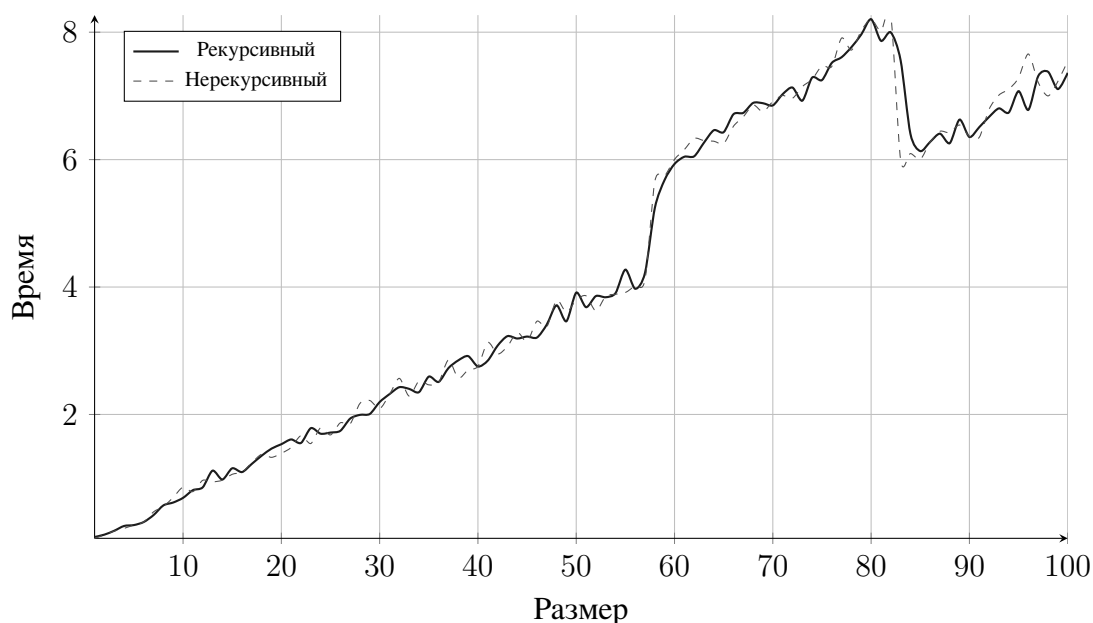


Рисунок 4.1 — График зависимости процессорного времени от N (при $k = 1000$).

Вывод

В данном разделе были проведены замеры процессорного времени для каждой реализации из предыдущего раздела. Полученные на основе этих замеров графики иллюстрируют рассчитанные ранее трудоёмкости рассматриваемых в работе алгоритмов.

Помимо этого, мной был проведён сравнительный анализ реализаций алгоритмов по критериям ёмкостной эффективности.

ЗАКЛЮЧЕНИЕ

В ходе работы мной были описаны рекурсивный и нерекурсивный алгоритмы, решающие одну и ту же задачу. Также мной была введена модель вычислений и с ее помощью вычислена трудоёмкость каждого из алгоритмов. Кроме того, были написаны реализации алгоритмов и выполнены замеры процессорного времени для лучшего и худшего случаев для каждой из реализаций.

Рассчитанная ранее теоретическая оценка трудоёмкости алгоритмов не соответствует полученным на практике значениям. Хотя в теории нерекурсивный алгоритм обладает наименьшей трудоёмкостью, на практике они выполняются примерно одинаковое время. Самым эффективным по критериям ёмкостной эффективности оказалась же реализация нерекурсивного алгоритма.

Таким образом, все задачи были выполнены, а поставленная цель достигнута.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Peter R. Recursive functions / Peter R. – Hungary, Academic Press, New York and London, 1967 – 308с. – Текст : непосредственный.
2. C17 standard (ISO/IEC 9899:2018) [стр. 288]. [Электронный ресурс] Режим доступа: <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2310.pdf> (дата обращения: 29.09.2025).